EMBEDDED / CONTROL / AUTONOMY / CASE STUDY

AEGIS AUTONOMY

CASE STUDY

02

A disciplined experiment workflow for testing whether a vehicle can detect lost state trust.

PROPOSED RESEARCH / NOT YET VERIFIED

THE BUSINESS CONTEXT

A small team studies autonomous vehicles whose position and attitude estimates can drift or fail through sensor bias and dropout. Today each fault study is a one off script. Results cannot be compared across estimators or monitors and false alarms are rarely counted alongside detection.

WHO IS INVOLVED AND WHAT THEY NEED

01	**Control researcher**	Wants fair comparison of monitors against a plain baseline controller.
02	**Safety reviewer**	Needs to see failure cases and false alarms rather than only successes.
03	**Programme lead**	Needs to know which claims the simulation can and cannot support.

SCOPE AND HONEST LIMITS

- ⚠ Simulation robustness is not hardware safety certification.
- ⚠ Detectability depends on observability and on the assumed sensor models.
- ⚠ No results exist yet and every target is something to measure.
- ⚠ Correlated faults may defeat all monitors tested.

CURRENT WORKFLOW AND FAILURE POINTS

02

PROPOSED RESEARCH / NOT YET VERIFIED

HOW THE WORK RUNS TODAY

1 TODAY

Ad hoc simulator

A script is edited for each fault study and parameters are changed by hand.

2 TODAY

Single run results

One trajectory is plotted and presented as typical behaviour.

3 TODAY

Mixed fault logic

Fault injection is tangled with dynamics so assumptions are hard to inspect.

4 TODAY

Thresholds by feel

Monitor thresholds are tuned until the demo looks good.

5 TODAY

Unrecorded settings

Seeds and software versions are not kept so runs cannot be repeated.

6 TODAY

Selective reporting

Runs where the monitor failed are dropped from the write up.

WHERE IT BREAKS

Unrepeatable runs	Nobody else can reproduce a result without the original machine state.
Hidden false alarms	Detection speed is shown while alarms on healthy runs are ignored.
Untested model limits	Estimator miscalibration and correlated faults are rarely explored.
Weak baselines	No plain controller is run under identical faults for comparison.

REDESIGNED WORKFLOW AND DECISION AUTHORITY

02

PROPOSED RESEARCH / NOT YET VERIFIED

THE REDESIGNED WORKFLOW

1 PERSON

Write the specification

State dynamics assumptions and fault classes and the safety boundary up front.

→

2 SYSTEM

Freeze the protocol

Fix monitors and baselines and metrics and seed sets before any run starts.

→

3 SYSTEM

Run seeded campaigns

Execute Monte Carlo runs across fault types with full configuration capture.

↓

4 SYSTEM

Score every run

Compute detection delay and false alarms and constraint violations automatically.

→

5 PERSON

Review negative cases

Inspect missed faults and unnecessary degradations and judge their cause.

→

6 SYSTEM

Publish with limits

Release configs and raw results and a brief on where the method fails.

SYSTEM STEP PERSON DECISION

WHERE AUTHORITY STAYS WITH A PERSON

Fault classes and safety boundary	CONTROL RESEARCHER	The scope sets what the results can mean and must be fixed before running.
Whether a failed case is excluded	SAFETY REVIEWER	Exclusion needs an independent person so failures stay in the report.
Wording of conclusions	PROGRAMME LEAD	Claims must stay within simulation evidence and not imply certification.

ARCHITECTURE, DATA AND CONTROLS

02

PROPOSED RESEARCH / NOT YET VERIFIED

SYSTEM ARCHITECTURE

1	**Vehicle dynamics**	Deterministic reduced order or six degree of freedom model with seeded noise.
2	**Estimator**	Error state or extended Kalman filter exposing covariance and residuals.
3	**Fault monitor**	Fixed threshold and innovation based and adaptive monitors behind one interface.
4	**Degraded controller**	Bounded fallback behaviour selected by a supervisory state machine.
5	**Evaluation harness**	Experiment runner that stores configs and raw traces and computed metrics.

CORE DATA ENTITIES

- **Scenario**
- **Fault injection**
- **Estimator state**
- **Monitor decision**
- **Run record**
- **Metric result**

RISKS AND THE CONTROL FOR EACH

Tuning to the test set →	Thresholds are fixed on a development seed set and scored on separate held out seeds.
Cherry picked trajectories →	Reports aggregate every run and list the worst cases beside the typical ones.
Simulation read as certification →	Every report states that results are simulation only and not hardware safety evidence.
Model mismatch unnoticed →	Estimator miscalibration is injected deliberately and its effect is reported.

EVALUATION AND DELIVERY PLAN

02

PROPOSED RESEARCH / NOT YET VERIFIED

HOW SUCCESS IS MEASURED

Detection delay	Time from fault onset to monitor flag measured per fault class and seed.
False alarm rate	Monitor flags raised per hour of healthy simulated operation.
Trajectory error	Deviation from the planned path compared with the baseline controller.
Boundary violations	Count of runs that cross the declared safety boundary.
Mission completion	Share of runs that finish the task safely under each fault.

TEST PLAN

1. **Rerun a stored campaign on a clean machine and confirm identical traces.**
2. **Inject no fault and confirm the monitor stays quiet beyond the stated rate.**
3. **Check that the baseline and monitored controllers see identical noise draws.**
4. **Swap in a deliberately wrong covariance and confirm the report shows the effect.**

DELIVERY PLAN

01 ALIGN	02 UNDERSTAND	03 DESIGN	04 DELIVER	05 LAUNCH
Agree the research question and the fault classes and what a useful answer looks like.	Review sensor and vehicle assumptions and existing monitoring methods.	Specify the simulator layers and metrics and the frozen experiment protocol.	Build the runner and execute campaigns and review the negative cases.	Publish configs and results and the brief on where the method fails.
DECISION Scope of faults studied	**DECISION** Which baselines are fair	**DECISION** Protocol sign off	**DECISION** How failures are classed	**DECISION** Which claims are released

DELIVERABLES

- Experiment specification and frozen protocol
- Reproducible experiment runner
- Baseline against monitored benchmark report
- Raw run traces and configuration sets
- Technical brief on failure cases